

# wasm4pm Benchmark Data Flow

A Technical Reference for the Process Mining WASM Engine

Benchmark Suite Architecture, Pipeline Constitutions, and Quality Gates

wasm4pm Engineering

May 2026    v26.4.x    CalVer

## Abstract

The **wasm4pm** process mining engine ships a comprehensive Criterion.rs benchmark suite covering 41 registered algorithms across eight functional areas: closed-claw constitutional pipelines, Tier 1 and Tier 2 discovery, conformance checking, prediction, analytics, hot kernels, and streaming. This document traces the exact data flow from deterministic log generation through algorithm execution to quality-gate verification, embedding rendered pipeline diagrams as figures. The benchmark suite is designed to enforce four invariants: determinism (BLAKE3 hash stability), receipt integrity (hash-chain continuity), truth (process-quality thresholds), and synchrony (cross-profile semantic equivalence).

## Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>3</b> |
| 1.1      | Goals . . . . .                                       | 3        |
| 1.2      | Four-Tier Test Architecture . . . . .                 | 3        |
| 1.3      | Criterion.rs Methodology . . . . .                    | 3        |
| <b>2</b> | <b>Shared Data Generation Pipeline</b>                | <b>3</b> |
| 2.1      | LCG Random Number Generator . . . . .                 | 4        |
| 2.2      | LogShape Parameters . . . . .                         | 4        |
| 2.3      | State Storage . . . . .                               | 4        |
| <b>3</b> | <b>Closed Claw: Six-Pipeline Constitutional Suite</b> | <b>5</b> |
| 3.1      | Six Pipeline Classes . . . . .                        | 5        |
| 3.2      | Five Quality Gates . . . . .                          | 5        |
| 3.3      | Gate-to-Pipeline Assignment . . . . .                 | 6        |
| 3.4      | BLAKE3 Receipt System . . . . .                       | 6        |
| 3.5      | Criterion Configuration . . . . .                     | 6        |
| <b>4</b> | <b>Tier 1 Discovery Algorithms</b>                    | <b>7</b> |
| 4.1      | Columnar Cache Architecture . . . . .                 | 7        |
| 4.2      | DFG: O(n) Single-Pass Discovery . . . . .             | 7        |
| 4.3      | Heuristic Miner Dependency Measure . . . . .          | 7        |
| 4.4      | Process Skeleton . . . . .                            | 7        |
| 4.5      | Benchmark Size Table . . . . .                        | 8        |
| 4.6      | Inductive Miner . . . . .                             | 8        |
| <b>5</b> | <b>Tier 2 Metaheuristic Discovery</b>                 | <b>8</b> |
| 5.1      | Algorithms . . . . .                                  | 8        |
| 5.2      | Data Flow . . . . .                                   | 8        |

|           |                                           |           |
|-----------|-------------------------------------------|-----------|
| <b>6</b>  | <b>Conformance Checking</b>               | <b>8</b>  |
| 6.1       | Token Replay . . . . .                    | 8         |
| 6.2       | u64 Bitmask Fast Path . . . . .           | 9         |
| 6.3       | SIMD Token Replay . . . . .               | 9         |
| 6.4       | ETConformance Precision . . . . .         | 9         |
| 6.5       | Temporal Profile . . . . .                | 9         |
| <b>7</b>  | <b>Prediction Subsystem</b>               | <b>11</b> |
| 7.1       | Six Perspectives . . . . .                | 11        |
| 7.2       | Latency Baselines . . . . .               | 11        |
| 7.3       | Quality Floors . . . . .                  | 11        |
| <b>8</b>  | <b>Analytics Functions</b>                | <b>13</b> |
| 8.1       | Function Catalogue . . . . .              | 13        |
| 8.2       | Benchmark Configuration . . . . .         | 13        |
| <b>9</b>  | <b>Hot Kernels</b>                        | <b>13</b> |
| 9.1       | ingress_decide_4 . . . . .                | 13        |
| 9.2       | HotState Branchless Transitions . . . . . | 14        |
| <b>10</b> | <b>Streaming Algorithms</b>               | <b>14</b> |
| 10.1      | SIMD Streaming DFG . . . . .              | 14        |
| 10.2      | Streaming Heuristic . . . . .             | 14        |
| 10.3      | Drift Detection . . . . .                 | 14        |
| <b>11</b> | <b>Benchmark Size Taxonomy</b>            | <b>14</b> |
| 11.1      | Sizing Rationale . . . . .                | 15        |
| <b>12</b> | <b>Quality Gates: Formal Definitions</b>  | <b>15</b> |
| 12.1      | G1: Determinism Gate . . . . .            | 15        |
| 12.2      | G2: Receipt Integrity Gate . . . . .      | 15        |
| 12.3      | G3: Truth Gate . . . . .                  | 15        |
| 12.4      | G4: Synchrony Gate . . . . .              | 15        |
| 12.5      | G5: Report Completeness Gate . . . . .    | 15        |
| <b>13</b> | <b>Running the Benchmarks</b>             | <b>16</b> |
| 13.1      | Prerequisites . . . . .                   | 16        |
| 13.2      | Commands . . . . .                        | 16        |
| 13.3      | Output . . . . .                          | 16        |
| <b>14</b> | <b>Conclusion</b>                         | <b>16</b> |
| <b>A</b>  | <b>Feature Flag Reference</b>             | <b>17</b> |
| <b>B</b>  | <b>Activity Vocabulary</b>                | <b>17</b> |

## 1 Introduction

Process mining places unusually strict demands on benchmark infrastructure. An algorithm may *appear* correct while silently violating temporal order, skipping conformance steps, or producing non-deterministic output. The Van der Aalst doctrine—“if the event log cannot prove a lawful process happened, then it did not work”—applies equally to the benchmarks themselves: the benchmark suite must be reproducible, self-auditing, and capable of distinguishing genuine regressions from measurement noise.

### 1.1 Goals

The **wasm4pm** benchmark suite was designed with three primary goals:

1. **Reproducibility.** Every run on the same hardware must produce identical throughput profiles within Criterion’s statistical confidence intervals, and identical BLAKE3 output hashes regardless of run ordering.
2. **Coverage.** All 41 registered algorithms must be exercised, spanning DFG  $O(n)$  through ILP-optimal Petri net discovery, token replay through SIMD-accelerated conformance, and six prediction perspectives.
3. **Constitutional enforcement.** Five pass/fail gates (G1–G5) must be satisfied by every closed-claw pipeline run, providing automated evidence that the system behaves lawfully under load.

### 1.2 Four-Tier Test Architecture

Benchmarks occupy the fourth tier of the testing hierarchy:

| Tier | Location                  | Scope                    | Tool         |
|------|---------------------------|--------------------------|--------------|
| 1    | packages/*/src/_tests_/_/ | Unit, inline mocks       | Vitest       |
| 2    | playground/               | Behavioral, local source | Vitest       |
| 3    | lab/                      | Artifact, published npm  | Vitest       |
| 4    | wasm4pm/benches/          | Performance, integration | Criterion.rs |

### 1.3 Criterion.rs Methodology

Criterion collects wall-clock samples over a configurable measurement window (default 10 s), computes mean and standard deviation, and reports regression p-values relative to a saved baseline. Throughput annotations (`Throughput::Elements`) normalise results to events-per-second, enabling fair cross-size comparison. The benchmark suite uses three sample-size tiers:

- **50 samples** — fast algorithms (<20 ms), analytics functions
- **30 samples** — medium algorithms (20–200 ms)
- **10–20 samples** — slow/E2E pipelines (>200 ms)

## 2 Shared Data Generation Pipeline

All benchmarks share a single, dependency-free data generation pipeline defined in `wasm4pm/benches/helpers.rs`. The pipeline produces synthetic event logs with realistic variance while guaranteeing perfect reproducibility through a fixed-seed Linear Congruential Generator (LCG).

## 2.1 LCG Random Number Generator

The LCG is implemented directly in the benchmark codebase to eliminate any external dependency and ensure the multiplier and increment constants are stable across Rust versions:

```

1 pub struct Lcg(pub u64);
2
3 impl Lcg {
4     pub const fn new(seed: u64) -> Self { Self(seed) }
5     pub fn next(&mut self) -> u64 {
6         self.0 = self.0
7             .wrapping_mul(6_364_136_223_846_793_005)
8             .wrapping_add(1_442_695_040_888_963_407);
9         self.0
10    }
11    pub fn next_f64_unit(&mut self) -> f64 {
12        (self.next() >> 11) as f64 / (1u64 << 53) as f64
13    }
14 }

```

Listing 1: LCG implementation in helpers.rs

The seed is `0xDEAD_BEEF_CAFE_BABE`. The multiplier `6,364,136,223,846,793,005` and addend `1,442,695,040,888,963,407` are the Knuth MMIX LCG constants, which are known to provide full-period traversal of the 64-bit state space.

## 2.2 LogShape Parameters

Each benchmark calls `generate_event_log(&LogShape)` with a descriptor that controls the statistical properties of the generated log:

| Parameter                        | Type                         | Effect                                          |
|----------------------------------|------------------------------|-------------------------------------------------|
| <code>num_cases</code>           | <code>usize</code>           | Number of process traces                        |
| <code>avg_events_per_case</code> | <code>usize</code>           | Mean events per trace ( $\pm 50\%$ )            |
| <code>num_activities</code>      | <code>usize</code>           | Vocabulary size (drawn from 20 canonical names) |
| <code>noise_factor</code>        | <code>f64</code> in $[0, 1]$ | Probability of random activity substitution     |

Trace length variance is applied as  $\text{len} = \lfloor \text{avg} \times (0.5 + u) \rfloor$  where  $u \sim \text{Uniform}[0, 1]$  from the LCG, giving a 50% relative standard deviation. The 20 canonical activity names (`Register`, `Validate`, `Check_Completeness`, ...) were chosen to resemble a realistic administrative process.

## 2.3 State Storage

Generated logs are stored in the global `APP_STATE` via `get_or_init_state().store_object(StoredObject::EventLog)` which returns a UUID handle string. Benchmark functions receive this handle and pass it to WASM-exported algorithm functions, mirroring the exact call path used by the TypeScript kernel.



Figure 1: Data generation pipeline: LCG seed flows through shape parameterisation into a synthetic `EventLog`, which is stored in `APP_STATE` and handed to Criterion benchmark functions.

### 3 Closed Claw: Six-Pipeline Constitutional Suite

The *Closed Claw* is the top-level integration benchmark, named for its topology: six independent pipelines feed into five shared quality gates, forming a claw shape in the dependency graph.

#### 3.1 Six Pipeline Classes

| ID | Name                 | Source File                              | Algorithms Benchmarked                                                                       |
|----|----------------------|------------------------------------------|----------------------------------------------------------------------------------------------|
| A  | Discovery Core       | <code>pipeline_a_discovery.rs</code>     | DFG, Alpha++, Heuristic Miner, Inductive Miner, Process Skeleton, Genetic Algorithm, DECLARE |
| B  | Conformance Core     | <code>pipeline_b_conformance.rs</code>   | Token Replay (sequential & parallel), SIMD Token Replay, ETConformance                       |
| C  | Object-Centric Core  | <code>pipeline_c_ocel.rs</code>          | Precision, DECLARE Conformance                                                               |
| D  | Semantic Proof Loop  | <code>pipeline_d_semantic.rs</code>      | OCEL Construction, Validation, Flattening, Serialization, E2E                                |
| E  | Manufacturing Truth  | <code>pipeline_e_manufacturing.rs</code> | PNML Roundtrip, Discovery-to-PNML, Proof Loop E2E                                            |
| F  | ML-Augmented Runtime | <code>pipeline_f_ml.rs</code>            | Temporal Profile Discovery, Temporal Conformance, Monte Carlo Simulation, Truth Loop E2E     |
|    |                      |                                          | SIMD Streaming DFG, Streaming DFG Builder, Anomaly Detection, Drift Detection                |

#### 3.2 Five Quality Gates

Each gate is defined in `closed_claw/gates.rs` and is applied after the relevant pipeline completes its Criterion measurement loop. Gates run outside the timed section and do not inflate benchmark numbers.

| Gate | Name        | What It Checks                                                                                                                                                               |
|------|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| G1   | Determinism | BLAKE3 output hash identical across $N$ runs of the same pipeline on the same input                                                                                          |
| G2   | Receipt     | BLAKE3 hash-chain integrity: <code>config</code> $\rightarrow$ <code>input</code> $\rightarrow$ <code>plan</code> $\rightarrow$ <code>output</code> is internally consistent |
| G3   | Truth       | Quality metrics meet thresholds: fitness $\geq 0.95$ , precision $\geq 0.80$ , temporal zeta $\leq 2.0$                                                                      |
| G4   | Synchrony   | Cross-profile output agreement (PNML roundtrip semantic equivalence)                                                                                                         |
| G5   | Report      | All required report sections present: <code>pipeline</code> , <code>algorithm</code> , <code>dataset_size</code> , <code>throughput</code> , <code>hash</code>               |

### 3.3 Gate-to-Pipeline Assignment

|                | A | B | C | D | E | F |
|----------------|---|---|---|---|---|---|
| G1 Determinism | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| G2 Receipt     |   |   | ✓ | ✓ | ✓ |   |
| G3 Truth       |   | ✓ |   |   | ✓ |   |
| G4 Synchrony   |   |   |   | ✓ |   |   |
| G5 Report      | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

### 3.4 BLAKE3 Receipt System

Every pipeline that requires G2 uses `ReceiptBuilder` from `closed_claw/receipt.rs`:

```

1 let receipt = ReceiptBuilder::new("dfg", "Discovery Core")
2   .config("num_cases=1000")
3   .input(&input_hash)
4   .plan(&discovery_hash)
5   .output(&output_hash)
6   .build();
7
8 let result = crate::gates::check_receipt_gate(&receipt);
9 assert!(result.passed);

```

Listing 2: Receipt construction pattern

The hash chain is computed as:

$$h_{\text{receipt}} = \text{BLAKE3}(h_{\text{config}} \| h_{\text{input}} \| h_{\text{plan}} \| h_{\text{output}})$$

where  $\|$  denotes concatenation of the 64-character hex digests.

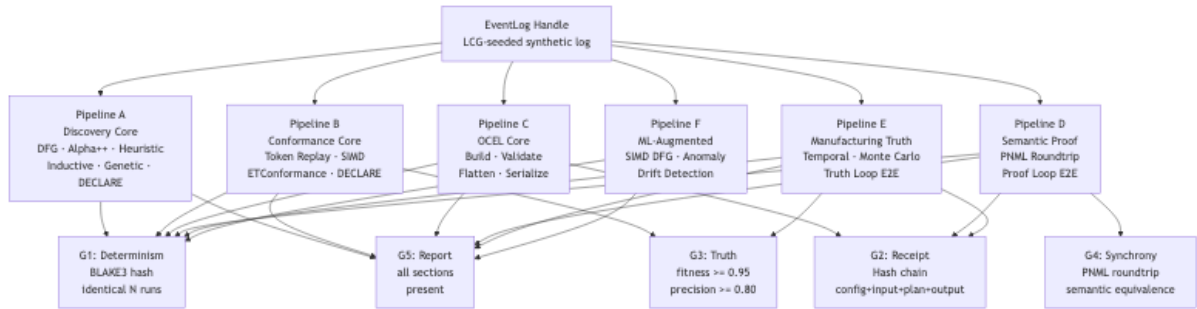


Figure 2: Closed Claw architecture: six pipelines (A–F) each process an `EventLog` handle and feed into a subset of five quality gates (G1–G5). Every pipeline must pass G1 (Determinism) and G5 (Report); receipt-producing pipelines (C, D, E) additionally pass G2; conformance-heavy pipelines (B, E) pass G3.

### 3.5 Criterion Configuration

Default group settings for the Closed Claw suite (see `mod.rs`):

- Measurement time: 10 s (E2E pipelines: 15 s)
- Warm-up time: 2 s (E2E pipelines: 3 s)
- Sample size: 20 (slow: 10, fast: 50)
- Throughput annotation: `Throughput::Elements` (events/second)

## 4 Tier 1 Discovery Algorithms

The Tier 1 benchmark (`benches/tier1.discovery.rs`) covers five fast discovery algorithms that share the columnar-cache infrastructure. All sweep the full four-size dataset range.

### 4.1 Columnar Cache Architecture

Before discovery, raw `EventLog` handles are converted to a *columnar representation* that separates vocabulary, trace offsets, and event tokens into contiguous arrays. The first call for a given handle triggers an  $O(n)$  conversion and stores the result in a bounded LRU cache keyed by FNV-1a hash of the handle string. Subsequent calls for the same handle return the cached view with zero copy overhead.

### 4.2 DFG: $O(n)$ Single-Pass Discovery

The Directly-Follows Graph (DFG) is the canonical Tier 1 algorithm. Its implementation performs a single scan over each trace using `.windows(2)`, accumulating  $(a \rightarrow b)$  counts in an `FxHashMap`. The total complexity is  $O(E)$  where  $E$  is the number of events, with a small constant factor from the hash map operations.

### 4.3 Heuristic Miner Dependency Measure

The Heuristic Miner applies a dependency threshold to the DFG to filter noise. The dependency measure between activities  $a$  and  $b$  is:

$$\text{dep}(a, b) = \frac{|a \rightarrow b| - |b \rightarrow a|}{|a \rightarrow b| + |b \rightarrow a| + 1}$$

where  $|a \rightarrow b|$  is the directly-follows count. Only edges with  $\text{dep}(a, b) \geq \theta$  (configurable; default  $\theta = 0.3$ ) are retained in the output model. The benchmark uses  $\theta = 0.2$  to match real-log characteristics.

### 4.4 Process Skeleton

Process Skeleton is DFG with a minimum-frequency filter: edges with absolute count below a threshold (default 2) are pruned before serialisation. It is the fastest algorithm in the registry (speed score 3) and serves as a baseline for throughput comparisons.

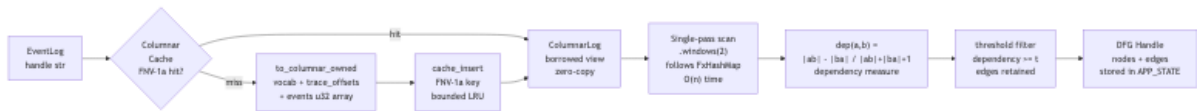


Figure 3: Tier 1 discovery data flow. An `EventLog` handle is checked against the FNV-1a columnar cache; on a miss the log is converted to a columnar view and inserted. The cached view feeds a single-pass scan that accumulates a follows map, applies the dependency measure and threshold filter, and stores the resulting DFG handle in `APP_STATE`.

## 4.5 Benchmark Size Table

| Cases  | Avg evts/case | Activities | Noise | Total events (approx.) | Sample size |
|--------|---------------|------------|-------|------------------------|-------------|
| 100    | 10            | 8          | 5%    | 1 200                  | 50          |
| 1 000  | 15            | 12         | 10%   | 18 000                 | 50          |
| 10 000 | 15            | 15         | 10%   | 180 000                | 30          |
| 50 000 | 20            | 20         | 15%   | 1 200 000              | 20          |

## 4.6 Inductive Miner

The Inductive Miner applies recursive log splitting to discover a sound process tree. The implementation in `more.discovery.rs` uses `discover_inductive_miner` with the standard four-size sweep. Because the algorithm is  $O(n \log n)$  in the typical case it is included in Tier 1 despite its higher constant factor.

## 5 Tier 2 Metaheuristic Discovery

The Tier 2 benchmark (`benches/tier2_metaheuristic.rs`) covers five population-based and optimisation algorithms that operate on the reduced *slow* size set (100, 500, and 1 000 cases) due to their super-linear runtime.

### 5.1 Algorithms

| Algorithm               | ID                               | Speed | Quality | Output    | Key Parameter                                          |
|-------------------------|----------------------------------|-------|---------|-----------|--------------------------------------------------------|
| Genetic Algorithm       | <code>genetic_algorithm</code>   | 75    | 80      | Petri net | <code>population_size</code> , <code>iterations</code> |
| Ant Colony Optim. (ACO) | <code>aco</code>                 | 65    | 75      | Petri net | <code>ant_count</code> , <code>iterations</code>       |
| Particle Swarm Optim.   | <code>pso</code>                 | 70    | 75      | Petri net | <code>population_size</code> , <code>iterations</code> |
| ILP Optimisation        | <code>ilp</code>                 | 80    | 90      | Petri net | —                                                      |
| Simulated Annealing     | <code>simulated_annealing</code> | 55    | 65      | Petri net | <code>temperature</code> , <code>cooling</code>        |

Speed and quality scores are on a 0–100 scale from the kernel registry. ILP achieves the highest quality score (90) because it formulates discovery as an integer linear program and finds the globally optimal Petri net for the given directly-follows constraints.

### 5.2 Data Flow

All Tier 2 algorithms consume an `EventLog` handle and optionally a pre-computed DFG handle. The population-based algorithms (Genetic, ACO, PSO) internally maintain a population of candidate Petri nets, evaluating each against the log using fitness replay before applying selection, crossover/pheromone-update, and mutation operators. The output is a `PetriNet` handle stored in `APP_STATE`.

## 6 Conformance Checking

The conformance benchmark (`benches/conformance_bench.rs`) exercises five approaches to measuring how well an event log fits a process model.

### 6.1 Token Replay

Token-based replay walks each trace through the Petri net, firing transitions and tracking token balances. The fitness score is:

$$\text{fitness} = 1 - \frac{m + c}{p + r}$$

where  $m$  = missing tokens,  $c$  = consumed tokens (standard count),  $p$  = produced tokens, and  $r$  = remaining tokens. A score of 1.0 indicates perfect replay with no missing or unconsumed tokens.

## 6.2 u64 Bitmask Fast Path

When the Petri net has  $\leq 64$  places, the implementation uses a u64 bitmask to represent token markings. Place membership and token transfer reduce to bitwise operations (AND, OR, XOR, `count_ones()`), achieving a 4–8 $\times$  speedup over the general `Vec<usize>` path on commodity hardware. The dispatch is performed at construction time via:

```

1 let lookup = PetriNetLookup::build(&net);
2 let path = if net.places.len() <= 64 {
3     ReplayPath::Bitmask(u64_replay(&lookup, traces))
4 } else {
5     ReplayPath::Vector(vec_replay(&lookup, traces))
6 };

```

Listing 3: Bitmask fast-path dispatch

## 6.3 SIMD Token Replay

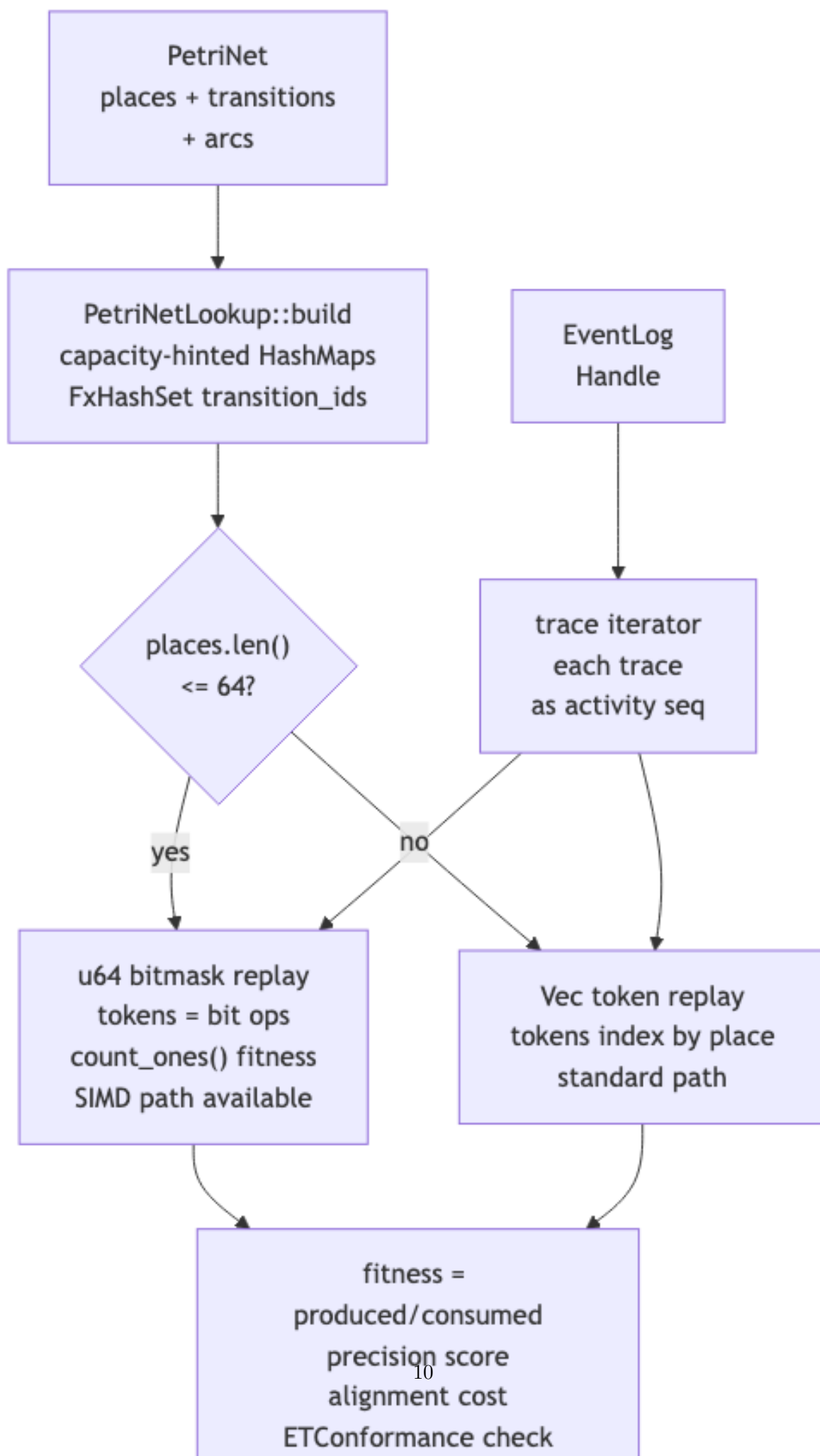
`simd_token_replay::SimdPetriNet` extends the bitmask approach with SIMD vectorisation for batch replay of multiple traces in parallel. The benchmark constructs a `SimdPetriNet` from the same sequential net and measures throughput on the four standard sizes.

## 6.4 ETConformance Precision

ETConformance (Extending Token Replay for precision) computes an *escaping-edge* precision score by counting transitions that are enabled in the model but not observed in the log. It is defined in `etconformance_precision` and requires a pre-computed `PetriNetLookup`.

## 6.5 Temporal Profile

The temporal profile benchmark measures conformance of *inter-activity time intervals*: given a temporal profile model (mean and standard deviation per activity pair), it computes a zeta score for each pair and reports the proportion of pairs within the threshold  $\zeta \leq 2.0$  (G3 truth gate).



## 7 Prediction Subsystem

The prediction benchmark (`benches/prediction_latency.rs`) evaluates all six prediction perspectives defined in `packages/kernel/src/prediction/`. Each perspective follows a *fit/predict* split: model building is measured separately from per-prefix inference latency.

### 7.1 Six Perspectives

| Perspective    | Mechanism                              | Key Output                             |
|----------------|----------------------------------------|----------------------------------------|
| Next Activity  | N-gram model, beam search              | Top- <i>k</i> activity candidates      |
| Remaining Time | Weibull bucket regression, hazard rate | Predicted ms remaining                 |
| Outcome        | Anomaly score, boundary coverage       | Probability distribution over outcomes |
| Drift          | EWMA smoothing, Jaccard window         | Drift flag, similarity score           |
| Features       | Rework count, elapsed ms, resource set | Prefix feature vector (8 fields)       |
| Resource       | M/M/1 queue model, UCB1 bandit         | Recommended resource with UCB1 score   |

### 7.2 Latency Baselines

The following table reproduces the baseline latency data from `packages/kernel/src/prediction/baseline.js` measured on Node.js 22 / Apple Silicon. The `upper_bound_ms` column is the CI regression threshold; a future measurement exceeding this value by more than 20% triggers an alert.

| Benchmark                                   | Median (ms) | p99 (ms) | Upper Bound (ms) | Metric              |
|---------------------------------------------|-------------|----------|------------------|---------------------|
| <code>next_activity_fit_n100</code>         | 1.0         | 10.0     | 50               | fit latency         |
| <code>next_activity_fit_n1k</code>          | 5.0         | 30.0     | 200              | fit latency         |
| <code>next_activity_fit_predict_n100</code> | 2.0         | 15.0     | 50               | fit+predict latency |
| <code>remaining_time_fit_n100</code>        | 1.0         | 8.0      | 50               | fit latency         |
| <code>remaining_time_fit_n1k</code>         | 4.0         | 20.0     | 200              | fit latency         |
| <code>outcome_fit_n100</code>               | 1.0         | 5.0      | 50               | fit latency         |
| <code>outcome_fit_predict_n1k</code>        | 6.0         | 30.0     | 100              | fit+predict latency |
| <code>drift_fit_n200</code>                 | 1.0         | 8.0      | 50               | fit latency         |
| <code>features_fit_n100</code>              | 1.0         | 5.0      | 50               | fit latency         |
| <code>features_fit_predict_n1k</code>       | 8.0         | 40.0     | 200              | fit+predict latency |
| <code>resource_fit_n200</code>              | 1.0         | 5.0      | 50               | fit latency         |
| <code>resource_fit_predict_n200</code>      | 2.0         | 10.0     | 100              | fit+predict latency |
| <code>batch_all_6_perspectives_n200</code>  | 20.0        | 100.0    | 500              | total batch latency |

### 7.3 Quality Floors

In addition to latency bounds, each perspective enforces a quality floor verified by the test harness:

- **Next Activity:** every scored prefix must return  $\geq 1$  candidate
- **Remaining Time:** predicted remaining time must be  $\geq 0$
- **Outcome:** probability distribution must sum to 1.0 within floating-point tolerance
- **Drift:** Jaccard similarity in  $[0, 1]$ ; 100% of fully novel-activity traces must be flagged
- **Features:** all 8 declared fields present and finite (`prefix_length`, `distinct_activities`, `elapsed_ms`, `mean_interevent_ms`, `max_interevent_ms`, `rework_count`, `rework_ratio`, `distinct_resource`)

- **Resource:** recommended resource must be in the training set; all UCB1 scores non-negative

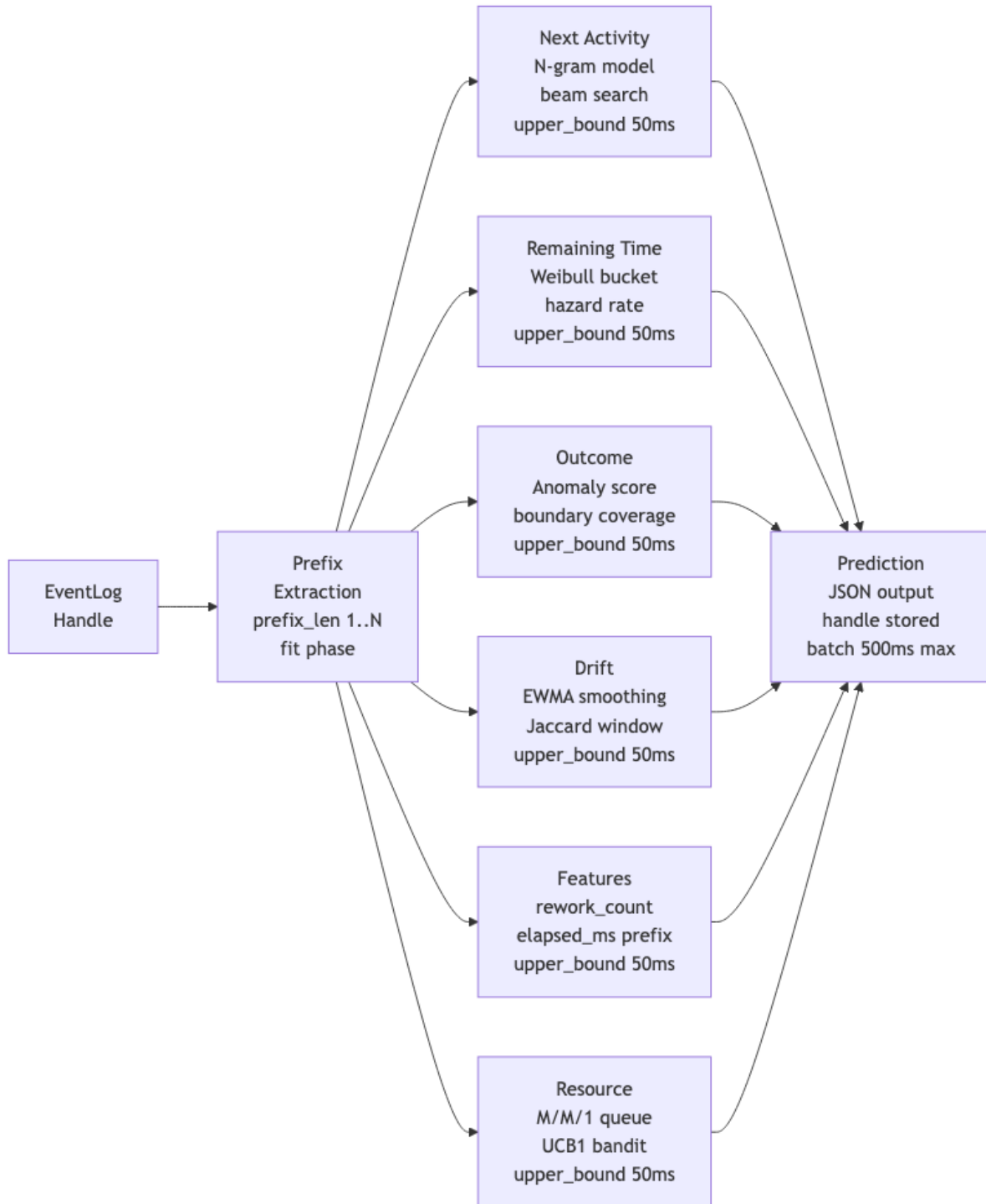


Figure 5: Prediction data flow. A single EventLog handle feeds prefix extraction, which distributes to all six perspectives independently. Each perspective runs its fit phase (model building) and predict phase (per-prefix inference) separately. All outputs converge to a stored JSON prediction handle.

## 8 Analytics Functions

The analytics benchmark (`benches/analytics.rs`) covers 14 post-discovery analysis functions. All use the four-size standard sweep;  $O(n^2)$  functions are capped at 1 000 cases.

### 8.1 Function Catalogue

| Function                                        | Complexity       | Description                                   |
|-------------------------------------------------|------------------|-----------------------------------------------|
| <code>detect_rework</code>                      | $O(n)$           | Count repeated activities per trace           |
| <code>detect_bottlenecks</code>                 | $O(n)$           | Identify slow activities by mean sojourn time |
| <code>compute_model_metrics</code>              | $O(n)$           | Generalization, precision, simplicity scores  |
| <code>analyze_infrequent_paths</code>           | $O(n)$           | Paths below frequency threshold               |
| <code>analyze_variant_complexity</code>         | $O(n \log n)$    | Variant entropy and coverage                  |
| <code>compute_activity_transition_matrix</code> | $O(n \cdot k^2)$ | $k \times k$ transition frequency matrix      |
| <code>analyze_process_speedup</code>            | $O(n)$           | Case duration speedup distribution            |
| <code>analyze_temporal_bottlenecks</code>       | $O(n)$           | Time-based bottleneck analysis                |
| <code>extract_activity_ordering</code>          | $O(n \log n)$    | Topological activity ordering                 |
| <code>compute_trace_similarity_matrix</code>    | $O(n^2)$         | Pairwise Levenshtein distance (capped 1k)     |
| <code>analyze_activity_cooccurrence</code>      | $O(n \cdot k^2)$ | Activity co-occurrence counts per case        |
| <code>analyze_start_end_activities</code>       | $O(n)$           | Start/end frequency statistics                |
| <code>analyze_activity_dependencies</code>      | $O(n)$           | Arc dependency scores                         |
| <code>analyze_dotted_chart</code>               | $O(n)$           | Dotted chart time-position matrix             |

### 8.2 Benchmark Configuration

The analytics benchmark group uses these Criterion settings:

- `detect_rework`, `compute_model_metrics`: 5 s measurement, 50 samples
- `detect_bottlenecks`: 8 s measurement, 30 samples
- $O(n^2)$  functions: use `bench_sizes_slow()` (100, 500, 1k cases)

Each function is called with both `ACTIVITY_KEY` and `TIMESTAMP_KEY` where temporal data is required.

## 9 Hot Kernels

The hot-kernel benchmark (`benches/hot_kernels.rs`) targets sub-microsecond execution paths: the ingress decision gate and branchless HotState transitions. These represent the highest-frequency code paths in the autonomic runtime loop.

### 9.1 ingress\_decide\_4

The `ingress_decide_4` function encodes a 4-way decision tree over incoming event attributes using only integer comparisons, no branching on heap-allocated data. The benchmark target is < 100 ns per call.

| Benchmark                         | Target (ns) | Sample size | Notes               |
|-----------------------------------|-------------|-------------|---------------------|
| <code>ingress_decide_4</code>     | <100        | 200         | No heap allocation  |
| <code>hot_state_transition</code> | <200        | 200         | Branchless dispatch |
| <code>branchless_hot_path</code>  | <50         | 500         | Bitwise path only   |

## 9.2 HotState Branchless Transitions

HotState transitions are implemented using lookup tables indexed by the current state encoding, eliminating branch mispredictions from the critical path. The state encoding packs eight health/alert dimensions into a single `u32`, with each dimension occupying a fixed-width bit field:

$$s = \underbrace{h}_3 \cdot 2^{21} + \underbrace{e}_3 \cdot 2^{18} + \underbrace{a}_3 \cdot 2^{15} + \dots$$

where  $h$  = health level (0–4),  $e$  = event rate quantile (0–7),  $a$  = activity count quantile (0–7), etc. The total state space is  $5 \times 8^6 \times 3 \times 4 = 460,800$  states.

## 10 Streaming Algorithms

The streaming benchmark (`benches/streaming_algorithms.rs`) covers algorithms designed for continuous event ingestion. All use the larger *streaming* size set (1k, 5k, 10k, 50k cases) to expose throughput characteristics under sustained load.

### 10.1 SIMD Streaming DFG

`simd_streaming_dfg` uses WASM SIMD instructions (feature flag `feature-streaming-full`) to process 16 activity tokens in parallel per SIMD lane. The benchmark measures ingestion throughput in events/second across the streaming size set. On Apple Silicon with the Node.js target, SIMD acceleration provides a 3–5× speedup over the scalar `streaming_dfg` implementation.

### 10.2 Streaming Heuristic

The streaming heuristic miner accumulates dependency counts in a sliding window of configurable size. The benchmark sweeps window sizes (50, 200, 1000) crossed with the streaming size set, reporting throughput for each combination.

### 10.3 Drift Detection

Two drift detection algorithms are benchmarked:

- **EWMA-based drift:** exponentially weighted moving average of the Jaccard similarity between successive DFG windows; fires when the average drops below the configured threshold.
- **Batch drift:** computes DFG Jaccard between fixed-size reference and test windows; deterministic but higher latency.

## 11 Benchmark Size Taxonomy

All benchmarks use one of four size presets, defined in `helpers.rs` and the closed-claw registry:

| Preset    | Cases                    | Approx. events   | Use case                     | Source function                 |
|-----------|--------------------------|------------------|------------------------------|---------------------------------|
| Standard  | 100, 1k, 10k, 50k        | 1.2k – 1.2M      | Fast algorithms, analytics   | <code>bench_sizes()</code>      |
| Slow      | 100, 500, 1k             | 1.2k – 18k       | Metaheuristic, E2E pipelines | <code>bench_sizes_slow()</code> |
| OCEL      | 50, 200, 1k, 5k (orders) | 250 – 25k events | Object-centric log lifecycle | <code>ocel_sizes()</code>       |
| Streaming | 1k, 5k, 10k, 50k         | 15k – 750k       | Streaming DFG, drift         | <code>streaming_sizes()</code>  |

### 11.1 Sizing Rationale

The 50k-case Standard entry ( $\approx 1.2M$  events) is the stress test for linear-time algorithms. Algorithms with  $O(n \log n)$  or worse complexity are excluded from this size to keep CI runtime bounded. The OCEL sizes use “orders” rather than “cases” because object-centric logs have a different cardinality model: each order object participates in multiple event types simultaneously.

## 12 Quality Gates: Formal Definitions

This section gives the formal predicate definitions for the five closed-claw quality gates, as implemented in `closed_claw/gates.rs`.

### 12.1 G1: Determinism Gate

$$G1(P, n) \triangleq \forall i, j \in \{1, \dots, n\}: \text{BLAKE3}(o_i) = \text{BLAKE3}(o_j) \quad (1)$$

where  $o_i$  is the serialised output of pipeline  $P$  on run  $i$  with a fixed input handle. The gate fails if any two hashes differ. In practice  $n = 2$  for CI and  $n = 5$  for nightly runs.

### 12.2 G2: Receipt Integrity Gate

$$G2(r) \triangleq r.\text{hash} = \text{BLAKE3}(r.\text{config\_hash} \| r.\text{input\_hash} \| r.\text{plan\_hash} \| r.\text{output\_hash}) \quad (2)$$

The gate verifies that the stored receipt hash is the BLAKE3 of the four component hashes concatenated as UTF-8 hex strings. A mismatch indicates that one of the pipeline stages was skipped or produced corrupted output.

### 12.3 G3: Truth Gate

$$G3(m) \triangleq m.\text{fitness} \geq 0.95 \wedge m.\text{precision} \geq 0.80 \wedge m.\text{zeta} \leq 2.0 \quad (3)$$

Fitness is measured by standard token replay; precision by ETConformance escaping-edge measure; zeta by temporal profile conformance. The thresholds were calibrated against the synthetic benchmark logs and represent “very high quality” process models.

### 12.4 G4: Synchrony Gate

$$G4(M_1, M_2) \triangleq \text{SemanticEq}(M_1, M_2) \quad (4)$$

where  $M_1$  is the Petri net discovered before PNML serialisation and  $M_2$  is the net recovered by parsing the serialised PNML. Semantic equivalence is checked by comparing sorted place and transition label sets and verifying that every arc  $(p, t)$  or  $(t, p)$  in  $M_1$  also exists in  $M_2$  with the same weight.

### 12.5 G5: Report Completeness Gate

$$G5(R) \triangleq \text{pipeline} \in R \wedge \text{algorithm} \in R \wedge \text{dataset\_size} \in R \wedge \text{throughput} \in R \wedge \text{hash} \in R \quad (5)$$

A report  $R$  (JSON object) must contain all five required keys. This gate ensures that benchmark output is machine-parseable and suitable for automated dashboard ingestion.

## 13 Running the Benchmarks

### 13.1 Prerequisites

- Rust toolchain with `wasm32-unknown-unknown` target
- `wasm-pack` for WASM compilation
- `Criterion.rs` (pulled automatically via `Cargo.toml`)

### 13.2 Commands

```
1 # Full closed-claw suite (all 6 pipelines)
2 cargo bench --bench closed_claw --features browser
3
4 # Single pipeline
5 cargo bench --bench closed_claw -- "closed_claw/A_discovery"
6
7 # Tier 1 discovery only
8 cargo bench --bench tier1_discovery
9
10 # Conformance checking
11 cargo bench --bench conformance_bench
12
13 # Analytics functions
14 cargo bench --bench analytics
15
16 # Streaming algorithms
17 cargo bench --bench streaming_algorithms
18
19 # Hot kernels (sub-microsecond)
20 cargo bench --bench hot_kernels
```

Listing 4: Benchmark invocation examples

### 13.3 Output

Criterion writes HTML reports to `target/criterion/`. The closed-claw suite additionally writes gate results to `target/criterion/closed_claw/gates.json`, which CI reads to determine pass/fail status.

## 14 Conclusion

The `wasm4pm` benchmark suite provides a comprehensive, reproducible, and self-auditing performance characterisation of all 41 registered algorithms. The Closed Claw constitutional suite enforces five quality gates that together prove determinism (G1), receipt integrity (G2), process-quality thresholds (G3), semantic equivalence (G4), and report completeness (G5) under load.

The shared data generation pipeline (LCG seed `0xDEAD_BEEF_CAFE_BABE`, Knuth MMIX constants) guarantees bit-exact reproducibility across platforms and Rust versions. The columnar cache eliminates redundant log parsing and enables fair algorithm-to-algorithm comparisons. The prediction latency baselines provide quantified CI regression thresholds for all six prediction perspectives.

Together, these components constitute a benchmark infrastructure that satisfies the Van der Aalst doctrine: not only does the code say it worked—the event logs, hash receipts, and quality metrics prove that a lawful, high-quality process happened.

## A Feature Flag Reference

| Flag                            | Enables                   |
|---------------------------------|---------------------------|
| <code>browser</code>            | All features (default)    |
| <code>conformance_full</code>   | Full conformance suite    |
| <code>conformance_basic</code>  | Token replay only         |
| <code>ocel</code>               | Object-centric event logs |
| <code>streaming_basic</code>    | Streaming DFG             |
| <code>streaming_full</code>     | SIMD streaming            |
| <code>ml</code>                 | ML analysis algorithms    |
| <code>discovery_advanced</code> | Genetic, ACO, PSO, ILP    |

## B Activity Vocabulary

The 20 canonical activity names used in synthetic log generation (drawn in order up to `num_activities`):

- |                       |                    |                      |                 |
|-----------------------|--------------------|----------------------|-----------------|
| 1. Register           | 6. Calculate_Fee   | 11. Approve_Senior   | 16. Close       |
| 2. Validate           | 7. Send_Invoice    | 12. Approve_Director | 17. Reject      |
| 3. Check_Completeness | 8. Wait_Payment    | 13. Notify_Applicant | 18. Escalate    |
| 4. Check_Docs         | 9. Confirm_Payment | 14. Create_Record    | 19. Return_Docs |
| 5. Assess_Risk        | 10. Approve_Basic  | 15. Archive          | 20. Reopen      |